

Theoretical minimum

Для подготовки к экзамену я составил подробные ответы на вопросы из раздела **Theoretical minimum** (Теоретический минимум), так как они являются наиболее конкретными проверочными вопросами. Также я дополнил ответы информацией по темам из **Exam program**, которые не вошли в список вопросов (например, детали архитектур CNN и методы оптимизации).

Ответы основаны на предоставленном конспекте лекций (ml-mipt basic) и дополнены общепринятыми теоретическими знаниями там, где конспект заканчивается (в основном темы Deep Learning).

Ответы на вопросы: Theoretical minimum (Теоретический минимум)

1. Постановка задачи обучения с учителем (Supervised learning)

Задача обучения с учителем заключается в восстановлении зависимости между объектами и их целевыми метками.

- **Дано:** Обучающая выборка $X = \{(x_i, y_i)\}_{i=1}^n$, где x_i — вектор признаков объекта, а y_i — правильный ответ (таргет) для этого объекта.
- **Задача:** Найти функцию (алгоритм) $f(x)$, которая для произвольного объекта x предсказывает ответ $\hat{y} = f(x)$, максимально близкий к истинному y , минимизируя функцию потерь $L(y, \hat{y})$ на новых данных.
- **Типы задач:** Регрессия (ответ — число), Классификация (ответ — метка класса).

2. Постановка задачи обучения без учителя (Unsupervised learning). Два примера.

Задача обучения без учителя заключается в поиске структуры или закономерностей в данных, где нет правильных ответов (таргетов).

- **Дано:** Выборка $X = \{x_i\}_{i=1}^n$ (только признаки).
- **Примеры задач:**
 1. **Кластеризация (Clustering):** Группировка объектов в кластеры так, чтобы внутри группы объекты были похожи, а между группами — различны (например, сегментация клиентов).
 2. **Снижение размерности (Dimensionality Reduction):** Сжатие описания данных с сохранением максимума информации, например, метод PCA (для визуализации или сжатия).

3. Что такое i.i.d. данные?

i.i.d. означает **Independent and Identically Distributed** (независимые и одинаково распределенные).

Это ключевое предположение в машинном обучении, которое гласит, что:

1. Все объекты в выборке получены из одного и того же вероятностного распределения (Identically Distributed).
2. Появление одного объекта никак не влияет на появление другого (Independent).
Без этого предположения мы не могли бы гарантировать, что модель, обученная на тренировочной выборке (Train), будет работать на тестовой (Test).

4. Как работает Наивный Байесовский классификатор? Почему он "наивный"?

Алгоритм основан на теореме Байеса:

$$P(y|x) = \frac{P(x|y)P(y)}{P(x)}$$

Мы выбираем класс y , для которого эта вероятность максимальна.

- **Почему наивный:** Он делает "наивное" предположение, что **все признаки независимы друг от друга** при условии класса. Это позволяет расписать правдоподобие как произведение:
 $P(x|y) = P(x^{(1)}|y) \cdot P(x^{(2)}|y) \cdot \dots \cdot P(x^{(d)}|y)$.
- Несмотря на грубое упрощение (в реальности признаки часто коррелируют), алгоритм работает на удивление хорошо, особенно на текстах.

5. Линейная регрессия для задачи минимизации MSE. Формула и производная.

- **Модель:** $f(x) = \langle w, x \rangle + w_0$ (или в матричном виде $Y = Xw$).
- **Функция потерь (MSE):** $Q(w) = \|Y - Xw\|^2 = (Y - Xw)^T(Y - Xw)$.
- **Производная по весам:**
Раскрываем скобки: $Q(w) = Y^T Y - 2w^T X^T Y + w^T X^T X w$.
Дифференцируем по w :

$$\nabla_w Q(w) = -2X^T Y + 2X^T X w$$

Приравнявая к 0, получаем нормальное уравнение: $w = (X^T X)^{-1} X^T Y$.

6. Шаг градиентного спуска. Как адаптировать для больших датасетов?

- **Шаг GD:** $w_{new} = w_{old} - \eta \cdot \nabla_w Q(w)$, где η — темп обучения (learning rate). Градиент считается как сумма градиентов по всем объектам выборки.
- **Для больших датасетов:** Считать сумму по миллионам объектов на каждом шаге долго. Используют **Stochastic Gradient Descent (SGD)** или **Mini-batch SGD**.
 - Вместо всего датасета берется один случайный объект или маленькая пачка (батч) объектов, и градиент считается только по ним. Это шумная, но быстрая оценка истинного градиента.

7. Что такое правдоподобие (Likelihood)? Где используется MLE?

- **Правдоподобие** $P(X|\theta)$ — это вероятность получить наблюдаемую выборку X при фиксированных параметрах модели θ . В отличие от вероятности, здесь переменной являются параметры, а данные фиксированы.
- **MLE (Maximum Likelihood Estimation)**: Метод максимального правдоподобия используется для поиска параметров модели. Мы ищем такие θ , при которых вероятность пронаблюдать наши данные максимальна.
- **Где используется**: Большинство функций потерь (MSE в регрессии, LogLoss/Cross-Entropy в классификации) выводятся из принципа максимизации правдоподобия.

8. Что такое кросс-валидация? Влияние числа фолдов.

Это метод оценки качества модели, при котором обучающая выборка разбивается на K частей (фолдов). Модель обучается K раз: каждый раз на $K - 1$ частях, а валидируется на оставшейся одной. Результаты усредняются.

- **Число фолдов**:
 - Малое K (напр. 2-3): Вычисления быстрые, но оценка смещенная (bias), так как обучаемся на меньшей части данных.
 - Большое K (напр. N - Leave-One-Out): Вычисления очень долгие, оценка несмещенная, но имеет высокую дисперсию (variance), так как модели очень похожи друг на друга. Обычно используют $K = 5$ или 10.

9. Переобучение (Overfitting) и недообучение (Underfitting). Как детектировать?

- **Underfitting**: Модель слишком простая, не может выучить закономерности. Высокая ошибка и на train, и на test.
- **Overfitting**: Модель слишком сложная, запомнила шум в обучающих данных, но не выучила общий закон. Низкая ошибка на train, но высокая на test.
- **Детекция**: Строить кривые обучения (learning curves).
 - Если ошибка на Train падает, а на Validation растет (или выходит на плато сильно выше Train) — это переобучение.
 - Если ошибки на Train и Validation высокие и близки друг к другу — это недообучение.

10. Параметры vs Гиперпараметры. Пример.

- **Параметры**: Внутренние переменные модели, которые настраиваются (выучиваются) автоматически в процессе обучения (по градиенту или формуле).
 - *Пример*: Веса w в линейной регрессии, веса в нейросети, пороги в узлах решающего дерева.
- **Гиперпараметры**: Внешние настройки, которые задает человек до начала обучения. Они не меняются градиентным спуском.

- *Пример:* Коэффициент регуляризации λ в линейных моделях, глубина дерева (`max_depth`), число соседей k в kNN, `learning rate`.

11. Что такое регуляризация? Разница L1 и L2.

Регуляризация — это добавление штрафа за сложность модели (величину весов) к функции потерь для борьбы с переобучением. $Q_{reg}(w) = Q(w) + \lambda R(w)$.

- **L2 (Ridge):** Штраф $\|w\|_2^2$. "Размазывает" веса, делая их малыми, но не нулевыми. Дифференцируема. Эквивалентна гауссовскому априорному распределению весов.
- **L1 (Lasso):** Штраф $\|w\|_1$. Приводит к **разреженности** (sparsity) — зануляет веса незначимых признаков. Используется для отбора признаков. Эквивалентна априорному распределению Лапласа.

12. Регуляризуют ли bias (свободный член w_0)? Почему?

Нет, обычно w_0 (bias) не регуляризуют.

- **Почему:** Свободный член отвечает за смещение гиперплоскости относительно начала координат (среднее значение таргета), а не за наклон или сложность зависимости. Если его штрафовать, модель будет принудительно "тянуться" к началу координат, что внесет ненужное смещение и ухудшит качество, не помогая при этом с переобучением.

13. Зачем нормализовать данные перед линейной моделью?

1. **Интерпретируемость весов:** Если признаки в разных масштабах (метры и миллиметры), веса будут иметь разные порядки, их нельзя сравнивать (и регуляризация L1/L2 будет штрафовать их несправедливо).
2. **Скорость сходимости:** Градиентный спуск сходится быстрее, если линии уровня функции потерь близки к окружностям (сферам), а это достигается при одинаковом масштабе признаков. Без нормализации линии уровня — вытянутые эллипсы.

14. Линейная классификация. Что такое отступ (margin)?

- **Задача:** Разделить классы гиперплоскостью $\langle w, x \rangle = 0$.
- **Отступ (Margin):** Величина $M_i = y_i \cdot \langle w, x_i \rangle$, где $y_i \in \{-1, 1\}$.
 - Если $M_i > 0$, классификатор дал верный ответ.
 - Если $M_i < 0$, произошла ошибка.
 - Абсолютное значение $|M_i|$ показывает уверенность классификатора (расстояние от объекта до разделяющей гиперплоскости).

15. Precision (Точность) и Recall (Полнота). Использование.

- **Precision (Точность):** Доля действительно позитивных объектов среди всех, кого модель назвала позитивными. $\frac{TP}{TP+FP}$. "Как сильно мы можем доверять классификатору, если он сказал '1'?". Важна, когда цена ложного срабатывания высока (спам-фильтр: нельзя удалять важные письма).

- **Recall (Полнота):** Доля найденных позитивных объектов среди всех существующих позитивных. $\frac{TP}{TP+FN}$. "Сколько всего объектов класса '1' мы нашли?". Важна, когда цена пропуска высока (диагностика рака).

16. Несбалансированная выборка (95% класс 1). Как корректировать метрики?

Ассигасу (доля правильных ответов) будет 95% даже у константной модели, что бесполезно.

- **Как корректировать:**
 1. Использовать **Precision, Recall, F1-score** (гармоническое среднее) или **ROC-AUC / PR-AUC**.
 2. **Взвешивание классов (Class weights):** В функцию потерь добавить веса, штрафую за ошибку на минорном классе сильнее (вес обратно пропорционален частоте класса).
 3. **Ресемплинг:** Oversampling (дублирование минорного класса) или Undersampling (выкидывание мажорного).

17. Что такое ROC AUC? Как построить ROC-кривую?

- **ROC AUC** — Area Under Curve (площадь под ROC-кривой). Мера качества упорядочивания объектов (ранжирования). Вероятность того, что случайный положительный объект получит оценку выше, чем случайный отрицательный.
- **Построение ROC:**
 1. Откладываем по осям FPR (False Positive Rate) и TPR (True Positive Rate/Recall).
 2. Сортируем объекты по предсказанной вероятности.
 3. Двигаем порог от 1 до 0. Если встречаем объект класса 1 — идем вверх (растет TPR), если класса 0 — вправо (растет FPR).

18. Logistic Loss (LogLoss). Связь с MLE.

- Функция потерь для логистической регрессии: $L = - \sum (y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i))$.
- **Связь с MLE:** Логистическая регрессия предсказывает вероятность $P(y|x)$ (распределение Бернулли). Если записать правдоподобие выборки и взять от него логарифм с минусом (Negative Log-Likelihood), мы получим в точности формулу Logistic Loss. То есть минимизация LogLoss — это максимизация правдоподобия.

19. SVM (Метод опорных векторов). Основная идея и функционал.

- **Идея:** Провести разделяющую гиперплоскость так, чтобы расстояние до ближайших объектов обучающей выборки (опорных векторов) было **максимальным**. Это называется максимизацией ширины полосы (margin).
- **Функционал:** Минимизируем $\|w\|^2$ (для максимизации ширины $\frac{2}{\|w\|}$) с условием, что все объекты классифицированы верно с зазором: $y_i(\langle w, x_i \rangle + b) \geq 1$.
- На практике (Soft Margin SVM) используют **Hinge Loss**: $L = \sum \max(0, 1 - M_i) + C\|w\|^2$.

20. Жадный алгоритм построения решающего дерева.

Алгоритм ID3/C4.5/CART:

1. В корне находится вся выборка.
2. Перебираем все признаки j и все возможные пороги t для разбиения $x^{(j)} \leq t$.
3. Для каждого варианта считаем критерий информативности (прирост информации, уменьшение Энтропии или Gini).
4. Выбираем разбиение с лучшим значением критерия.
5. Разбиваем выборку на две части (левую и правую).
6. Рекурсивно повторяем для каждой ветки, пока не сработает критерий остановки (макс. глубина, мин. число объектов).

21. Почему дерево без ограничений дает 0 ошибок на уникальной выборке?

Если нет ограничений на глубину, дерево может строить ветвления до тех пор, пока в каждом листе не останется ровно по одному объекту (или по несколько с одинаковыми таргетами). В таком случае прогноз для обучающей выборки будет идеальным. Однако это ведет к **сильному переобучению** (дерево просто "запомнило" данные).

22. Как назначить метку класса в листе дерева?

Метка класса определяется голосованием большинства (Majority Vote): выбирается тот класс, объектов которого больше всего попало в этот лист в процессе обучения. Также можно выдавать вероятность (доля объектов класса в листе).

23. Как назначить значение в листе дерева регрессии? Зависит ли от критерия?

- Значение в листе — это константа c , минимизирующая функцию потерь в этом листе.
- **Зависимость:** Да.
 - Для **MSE** (среднеквадратичная ошибка) оптимальное предсказание — **среднее арифметическое** таргетов в листе.
 - Для **MAE** (средняя абсолютная ошибка) оптимальное предсказание — **медиана** таргетов в листе.

24. Что такое бэггинг (Bagging)?

Bootstrap Aggregating. Метод построения ансамбля, где:

1. Обучаются N независимых базовых моделей (обычно деревьев).
2. Каждая модель учится на своей подвыборке, полученной методом **бутстрапа** (выбор объектов из исходного датасета с возвращением).
3. Итоговый ответ — усреднение (регрессия) или голосование (классификация).
Бэггинг снижает **Variance** (разброс) модели, борется с переобучением.

25. Random Forest. Отличие от Бэггинга над деревьями.

Random Forest — это бэггинг над решающими деревьями с одним важным улучшением для **декорреляции** деревьев:

- В бэггинге при построении дерева перебираются все признаки для поиска лучшего сплита.
- В **Random Forest** в каждой вершине дерева выбирается случайное подмножество признаков (обычно $\sqrt{d}$), и лучший сплит ищется только среди них. Это делает деревья более разными (независимыми), что еще сильнее снижает разброс ансамбля.

26. Как обучаются базовые алгоритмы в градиентном бустинге?

Алгоритмы строятся **последовательно**.

1. Первый алгоритм учится предсказывать таргет.
2. Каждый следующий алгоритм $b_t(x)$ учится предсказывать **антиградиент функции потерь** (ошибку, сдвиг, "остатки") текущей композиции предыдущих алгоритмов: $s_i = -\nabla L(y_i, a_{t-1}(x_i))$.
3. Новый алгоритм добавляется к ансамблю с небольшим весом (learning rate): $a_t(x) = a_{t-1}(x) + \eta b_t(x)$.

27. Backpropagation в нейросетях. Векторные производные.

- **Backprop**: Это применение цепного правила дифференцирования (Chain Rule) от конца сети к началу.
 1. Прямой проход (Forward pass): вычисляем выходы слоев и Loss.
 2. Обратный проход (Backward pass): Сначала считаем градиент $\frac{\partial L}{\partial output}$. Затем передаем градиент назад, умножая на локальные производные функций активации и весов ($\frac{\partial output}{\partial input}$).
- **Векторная производная**: $\frac{\partial(Ax)}{\partial x} = A^T$. $\frac{\partial(x^T Ax)}{\partial x} = (A + A^T)x$. В backprop градиент — это вектор (или тензор) той же размерности, что и веса.

28. Как работает сверточный слой (Convolutional layer)?

Сверточный слой использует набор фильтров (ядер) — матриц малого размера (напр., 3x3).

- **Операция свертки**: Фильтр "скользит" по изображению с заданным шагом (stride). В каждой позиции производится поэлементное умножение значений фильтра на пиксели изображения и суммирование.
- Результат — карта признаков (feature map).
- Позволяет выделять локальные паттерны (границы, текстуры) и обеспечивает инвариантность к сдвигу.

29. Почему полносвязные (Dense) сети плохи для изображений? Почему CNN лучше?

1. **Количество параметров**: Для картинки 1000x1000 полносвязный слой к 1000 нейронам потребует $10^6 \times 10^3 = 10^9$ весов. Это невозможно обучить (переобучение, память). В CNN веса (ядро) общие для всего изображения (weight sharing).
2. **Потеря топологии**: Dense слой вытягивает картинку в вектор, теряя информацию о том, какие пиксели были рядом (пространственная структура). CNN работает с "соседями"

пикселя.

30. Как работает Vanilla RNN?

Рекуррентная сеть обрабатывает последовательности $x_1, x_2, \dots$.

У неё есть скрытое состояние h_t (память), которое зависит от текущего входа x_t и предыдущего состояния h_{t-1} :

$$h_t = \tanh(W_h h_{t-1} + W_x x_t + b)$$

$$y_t = W_y h_t$$

Проблема Vanilla RNN — затухание или взрыв градиентов при длинных последовательностях, из-за чего она "забывает" давний контекст.

31. Как работает Dropout?

Метод регуляризации нейросетей.

- Во время **обучения** каждый нейрон с вероятностью p (обычно 0.5) "выключается" (обнуляется его выход). Это предотвращает коадаптацию нейронов (чтобы они не полагались слишком сильно друг на друга) и имитирует обучение ансамбля множества прореженных сетей.

32. Поведение Dropout и Batch Normalization на этапе Inference (предсказания).

- Dropout:** На этапе предсказания **выключается** (все нейроны работают). Чтобы сохранить матожидание сигнала, веса умножаются на коэффициент вероятности p (или выходы делятся на $1 - p$ во время обучения — inverted dropout).
- Batch Normalization:** Во время обучения нормализует по текущему батчу (среднее и дисперсия батча). На этапе **inference** используются накопленные (running mean/var) скользящие средние и дисперсии, посчитанные за всё время обучения. Статистики батча одного тестового объекта не используются.

33. Постановка задачи PCA (Principal Component Analysis).

Задача: Найти такое ортогональное преобразование пространства признаков, чтобы в новой системе координат:

- Данные имели **максимальную дисперсию** вдоль осей (компонент).
- Ошибка реконструкции данных при отбрасывании наименее значимых компонент была **минимальна**.

Математически сводится к поиску собственных векторов ковариационной матрицы данных (SVD разложение).

Дополнения по программе экзамена (разделы, не покрытые вопросами)

Раздел 21. Матричное исчисление (примеры производных):

- $\frac{\partial}{\partial x}(a^T x) = a$
- $\frac{\partial}{\partial x}(x^T A x) = (A + A^T)x$ (если A симметрична, то $2Ax$)
- Якобиан (вектор по вектору): Если $y = f(x)$, где $y \in \mathbb{R}^m$, $x \in \mathbb{R}^n$, то матрица производных $J_{ij} = \frac{\partial y_i}{\partial x_j}$.

Раздел 26. Оптимизаторы в Deep Learning:

- **SGD Momentum:** Добавляем инерцию. Накапливаем вектор скорости $v_t = \gamma v_{t-1} + \eta \nabla L$. Шаг: $w = w - v_t$. Помогает проходить локальные минимумы и ускоряется в "оврагах".
- **RMSProp:** Адаптивный шаг. Делим градиент на корень из скользящего среднего квадратов градиентов. Нормализует шаг по разным направлениям (где крутой спуск — замедляет, где пологий — ускоряет).
- **Adam:** Комбинация Momentum и RMSProp. Хранит и инерцию градиента (mean), и инерцию квадрата градиента (variance). Стандарт де-факто в DL.

Раздел 28-29. Доп. по RNN и LSTM/GRU:

- **Проблема затухания градиента:** В Vanilla RNN при backpropagation through time (BPTT) градиенты умножаются на матрицу весов W много раз. Если собственные числа $W < 1$, градиент стремится к 0 (забываем начало).
- **LSTM/GRU:** Вводят механизм "ворот" (gates).
 - **LSTM:** Input gate, Forget gate, Output gate + cell state C_t (конвейер для проброса градиента без изменений).
 - **GRU:** Упрощенная версия (Reset gate, Update gate).

Раздел 31. Архитектуры CNN:

- **AlexNet:** Первая глубокая CNN (2012). Использовала ReLU, Dropout, MaxPooling.
- **VGG:** Использование только маленьких сверток 3x3. Глубокая и однородная структура.
- **Inception (GoogLeNet):** Идея Inception block — параллельное применение сверток разного размера (1x1, 3x3, 5x5) и конкатенация результатов. Позволяет извлекать признаки разного масштаба. Использование 1x1 сверток для снижения размерности каналов.
- **ResNet:** Skip connections (связи через слой). Слой учит не $H(x)$, а остаток $F(x) = H(x) - x$. Позволяет обучать очень глубокие сети (100+ слоев) без затухания градиента.

Раздел 32. Геометрические методы и кластеризация:

- **k-means:** Центроидный метод. 1. Инициализируем центры. 2. Относим точки к ближайшему центру. 3. Пересчитываем центры как среднее. Повторяем. Минимизирует внутрикластерную дисперсию. Линейные границы.

- **DBSCAN:** Плотностной метод. Ищет плотные сгустки точек. Параметры: радиус ϵ и мин. кол-во соседей. Умеет находить кластеры произвольной формы и выбросы (шум).
- **t-SNE:** Нелинейное снижение размерности для визуализации (Manifold Learning). Пытается сохранить локальные расстояния между точками (соседей оставить соседями) при переходе из многомерного пространства в 2D/3D. Использует t-распределение Стьюдента.
- **IsoMap:** Строит граф соседей, ищет кратчайшие пути (геодезическое расстояние) и применяет к этой матрице расстояний MDS (Multidimensional Scaling).